

從 Cloud Run 連線至全代管資料庫

來源：<https://codelabs.developers.google.com/connecting-to-serverless-databases-from-cloud-run?hl=zh-tw>

步驟數：11

整合無伺服器資料庫與在 Cloud Run 中運作的應用程式，並將服務設定為使用 Cloud Spanner 關聯資料庫和 Cloud Firestore

目錄

1. 總覽
2. 學習目標
3. 設定和需求
4. 建立及設定 [Spanner 執行個體](#)
5. 整合 [Spanner](#)
6. [Spanner](#) 概念
7. 建立及設定 [Firestore 資料庫](#)
8. [Firestore](#) 概念
9. 將 [Firestore](#) 整合至應用程式
10. 部署及測試
11. 恭喜！

步驟 1 · 約 0 分鐘

1. 總覽

在本實驗室中，您會將無伺服器資料庫(Spanner 和 Firestore) 與在 Cloud Run 中執行的應用程式(Go 和 Node.js) 整合。Cymbal Eats 應用程式包含多項服務，這些服務會在 Cloud Run 上執行。在下列步驟中，您將設定服務，以使用 [Cloud Spanner](#) 關聯式資料庫和 [Cloud Firestore](#) (NoSQL 文件資料庫)。使用無伺服器產品處理資料層和應用程式執行階段，可省去所有基礎架構管理工作，讓您專心建構應用程式，不必擔心額外負荷。

步驟 2 · 約 0 分鐘

2. 學習目標

在本實驗室中，您將瞭解如何執行下列操作：

- 整合 Spanner
 - 啟用 Spanner 代管服務
 - 整合至程式碼
 - 部署連線至 Spanner 的程式碼
 - 整合 Firestore
 - 啟用 Firestore Managed Services
 - 整合至程式碼
 - 部署連線至 Firestore 的程式碼
-

3. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The screenshot shows the Google Cloud Platform interface. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown menu, which is circled in blue. Below this, the 'Select a project' section includes a search bar and a 'NEW PROJECT' button, which is highlighted with a blue squiggle. The 'New Project' form is shown below, with fields for 'Project name' (my-kubernetes-codelab) and 'Project ID' (my-kubernetes-codelab). The Project ID field is circled in blue. There's also a 'Location' field set to 'No organization' with a 'BROWSE' button. At the bottom, there are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

設定環境

1. 建立專案 ID 變數

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
export SPANNER_INSTANCE=inventory-instance
export SPANNER_DB=inventory-db
export REGION=us-east1
export
SPANNER_CONNECTION_STRING=projects/$PROJECT_ID/instances/$SPANNER_INSTANCE/databases/$SPANNER
_DB
```

2. 啟用 Spanner、Cloud Run、Cloud Build 和 Artifact Registry API

```
gcloud services enable \
  compute.googleapis.com \
  spanner.googleapis.com \
  run.googleapis.com \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  firestore.googleapis.com \
  appengine.googleapis.com \
  artifactregistry.googleapis.com
```

3. 複製存放區

```
git clone https://github.com/GoogleCloudPlatform/cymbal-eats.git
```

4. 前往目錄

```
cd cymbal-eats/inventory-service/spanner
```

步驟 4 · 約 0 分鐘

4. 建立及設定 Spanner 執行個體

Spanner 是庫存服務後端的關聯式資料庫。您將在下列步驟中建立 Spanner 執行個體、資料庫和結構定義。

建立執行個體

1. 建立 Cloud Spanner 執行個體

```
gcloud spanner instances create $SPANNER_INSTANCE --config=regional-${REGION} \
--description="Cymbal Menu Inventory" --nodes=1
```

輸出範例

```
Creating instance...done.
```

2. 確認 Spanner 執行個體設定是否正確

```
gcloud spanner instances list
```

輸出範例

```
NAME: inventory-instance
DISPLAY_NAME: Cymbal Menu Inventory
CONFIG: regional-us-east1
NODE_COUNT: 1
PROCESSING_UNITS: 100
STATE: READY
```

建立資料庫和結構定義

建立新資料庫，並使用 [Google 標準 SQL 的資料定義語言 \(DDL\)](#) 建立資料庫結構定義。

1. 建立 DDL 檔案

```
echo "CREATE TABLE InventoryHistory (ItemRowID STRING (36) NOT NULL, ItemID INT64 NOT NULL,
InventoryChange INT64, Timestamp TIMESTAMP) PRIMARY KEY(ItemRowID)" >> table.ddl
```

2. 建立 Spanner 資料庫

```
gcloud spanner databases create $SPANNER_DB \
--instance=$SPANNER_INSTANCE \
--ddl-file=table.ddl
```

輸出範例

```
Creating database...done.
```

驗證資料庫狀態和結構定義

1. 查看資料庫狀態

```
gcloud spanner databases describe $SPANNER_DB \  
--instance=$SPANNER_INSTANCE
```

輸出範例

```
createTime: '2022-04-22T15:11:33.559300Z'  
databaseDialect: GOOGLE_STANDARD_SQL  
earliestVersionTime: '2022-04-22T15:11:33.559300Z'  
encryptionInfo:  
- encryptionType: GOOGLE_DEFAULT_ENCRYPTION  
name: projects/cymbal-eats-7-348013/instances/menu-inventory/databases/menu-inventory  
state: READY  
versionRetentionPeriod: 1h
```

注意：資料庫狀態會顯示為「READY」

2. 查看資料庫的結構定義

```
gcloud spanner databases ddl describe $SPANNER_DB \  
--instance=$SPANNER_INSTANCE
```

輸出範例

```
CREATE TABLE InventoryHistory (  
  ItemRowID STRING(36) NOT NULL,  
  ItemID INT64 NOT NULL,  
  InventoryChange INT64,  
  TimeStamp TIMESTAMP,  
) PRIMARY KEY(ItemRowID);
```

注意：資料庫有四個資料欄。ItemRowID 是主鍵。

您也可以在 [Spanner 總覽控制台](#) 中查看詳細資料。

5. 整合 Spanner

在本節中，您將瞭解如何將 Spanner 整合至應用程式。此外，SQL Spanner 提供[用戶端程式庫](#)、[JDBC 驅動程式](#)、[R2DBC 驅動程式](#)、[REST API](#) 和 [RPC API](#)，可讓您將 Spanner 整合至任何應用程式。

在下一節中，您將使用 Go 用戶端程式庫，在 Spanner 中安裝、驗證及修改資料。

安裝用戶端程式庫

[Cloud Spanner 用戶端程式庫](#)會自動使用應用程式預設憑證 (ADC) 尋找服務帳戶憑證，讓您更輕鬆地與 Cloud Spanner 整合

注意：更新程式碼時，範例程式碼會出現錯誤。您可以忽略這些錯誤。

設定驗證方法

Google Cloud CLI 和 Google Cloud 用戶端程式庫會自動偵測是否在 Google Cloud 上執行，並使用目前 Cloud Run 修訂版本的執行階段服務帳戶。這項策略稱為「應用程式預設憑證」，可讓程式碼在多個環境中攜帶使用。

不過，最佳做法是為函式指派由使用者管理的服務帳戶，而非預設服務帳戶，藉此建立專屬身分。

1. 將 Spanner 資料庫管理員角色授予服務帳戶

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
--member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
--role="roles/spanner.databaseAdmin"
```

輸出範例

```
Updated IAM policy for project [cymbal-eats-6422-3462].
[...]
```

服務帳戶擁有 Spanner 資料庫管理員角色後，即可執行下列操作：

- 取得/列出專案中所有 Cloud Spanner 執行個體。
- 在執行個體中建立/列出/捨棄資料庫。
- 授予/撤銷專案資料庫的存取權限。
- 讀取和寫入專案中所有 Cloud Spanner 資料庫。

使用用戶端程式庫

Spanner 用戶端程式庫可簡化與 Spanner 的整合作業，並支援多種常見的程式設計語言。

建立 Spanner 用戶端

Spanner 用戶端是用於讀取資料和將資料寫入 Cloud Spanner 資料庫的用戶端。除了 Close 方法外，用戶端可安全地同時使用。

以下程式碼片段會建立 Spanner 用戶端

main.go

```
var dataClient *spanner.Client
...
dataClient, err = spanner.NewClient(ctx, databaseName)
```

您可以將 Client 視為資料庫連線，所有與 Cloud Spanner 的互動都必須透過 Client。一般而言，您會在應用程式啟動時建立用戶端，接著重複使用該用戶端進行讀取、寫入及執行交易。每個用戶端都會使用 Cloud Spanner 中的資源。

修改資料

您可以透過多種方式，在 Spanner 資料庫中插入、更新及刪除資料。以下列出可用的方法。

- [Google Cloud Console](#)
- [gcloud CLI](#)
- [DML](#)
- [突變](#)

在本實驗室中，您將使用變異來修改 Spanner 中的資料。

Spanner 中的異動

Mutation 是變異作業的容器。Mutation 代表一系列的插入、更新和刪除作業，Cloud Spanner 會以不可分割的形式，將這些作業套用至 Cloud Spanner 資料庫中不同的資料列和資料表。

main.go

```
m := []*spanner.Mutation{}

m = append(m, spanner.Insert(
    "inventoryHistory",
    inventoryHistoryColumns,
    []interface{}{uuid.New().String(), element.ItemID, element.InventoryChange,
    time.Now()}))
```

這段程式碼片段會將新資料列插入商品目錄記錄資料表。

部署及測試

Spanner 設定完成並檢查過重要程式碼元素後，即可將應用程式部署至 Cloud Run。

將應用程式部署至 Cloud Run

Cloud Run 只要一個指令，就能自動建構、推送及部署程式碼。在下列指令中，您將在 `run` 服務上呼叫 `deploy` 指令，並傳遞執行中應用程式使用的變數，例如您先前建立的 `SPANNER_CONNECTION_STRING`。

1. 按一下「開啟終端機」
2. 將庫存服務部署至 Cloud Run

```
gcloud run deploy inventory-service \
  --source . \
  --region $REGION \
  --update-env-vars SPANNER_CONNECTION_STRING=$SPANNER_CONNECTION_STRING \
  --allow-unauthenticated \
  --project=$PROJECT_ID \
  --quiet
```

輸出範例

```
Service [inventory-service] revision [inventory-service-00001-sug] has been deployed
and is serving 100 percent of traffic.
Service URL: https://inventory-service-ilwytgcbca-uk.a.run.app
```

注意：如果系統提示您繼續操作，請輸入「Y」

3. 儲存服務網址

```
INVENTORY_SERVICE_URL=$(gcloud run services describe inventory-service \
  --platform managed \
  --region $REGION \
  --format=json | jq \
  --raw-output ".status.url")
```

測試 Cloud Run 應用程式

插入項目

1. 在 Cloud Shell 中輸入下列指令。

```
POST_URL=$INVENTORY_SERVICE_URL/updateInventoryItem
curl -i -X POST ${POST_URL} \
  --header 'Content-Type: application/json' \
  --data-raw '[
  {
    "itemID": 1,
    "inventoryChange": 5
  }
]'
```

輸出範例

```
HTTP/2 200
access-control-allow-origin: *
content-type: application/json
x-cloud-trace-context: 10c32f0863d26521497dc26e86419f13;o=1
date: Fri, 22 Apr 2022 21:41:38 GMT
server: Google Frontend
content-length: 2
```

OK

查詢項目

1. 查詢商品目錄服務

```
GET_URL=$INVENTORY_SERVICE_URL/getAvailableInventory
curl -i ${GET_URL}
```

回應範例

```
HTTP/2 200
access-control-allow-origin: *
content-type: text/plain; charset=utf-8
x-cloud-trace-context: b94f921e4c2ae90210472c88eb05ace8;o=1
date: Fri, 22 Apr 2022 21:45:50 GMT
server: Google Frontend
content-length: 166

[{"ItemID":1,"Inventory":5}]
```

6. Spanner 概念

Cloud Spanner 使用宣告式 SQL 陳述式查詢資料庫。SQL 陳述式會指出使用者想要的內容，不會說明如何取得結果。

1. 在終端機中輸入這項指令，查詢先前建立的記錄。

```
gcloud spanner databases execute-sql $SPANNER_DB \  
--instance=$SPANNER_INSTANCE \  
--sql='SELECT * FROM InventoryHistory WHERE ItemID=1'
```

輸出範例

```
ItemRowID: 1  
ItemID: 1  
InventoryChange: 3  
Timestamp:
```

查詢執行計畫

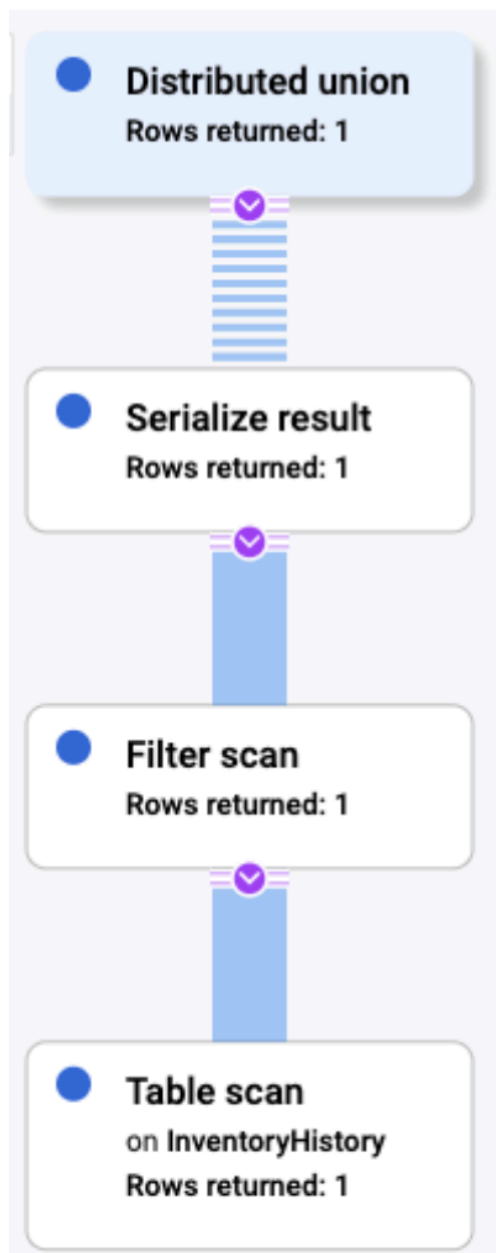
「查詢執行計畫」是 Spanner 用來取得結果的一連串步驟。取得特定 SQL 陳述式結果的方式可能有很多種。您可以在控制台和用戶端程式庫中存取查詢執行計畫。如要瞭解 Spanner 如何處理 SQL 查詢，請按照下列步驟操作：

1. 在控制台中開啟 Cloud Spanner 執行個體頁面。
2. 前往 Cloud Spanner 執行個體
3. 按一下 Cloud Spanner 執行個體的名稱。在資料庫部分，選取要查詢的資料庫。
4. 按一下「查詢」。
5. 在查詢編輯器中輸入下列查詢

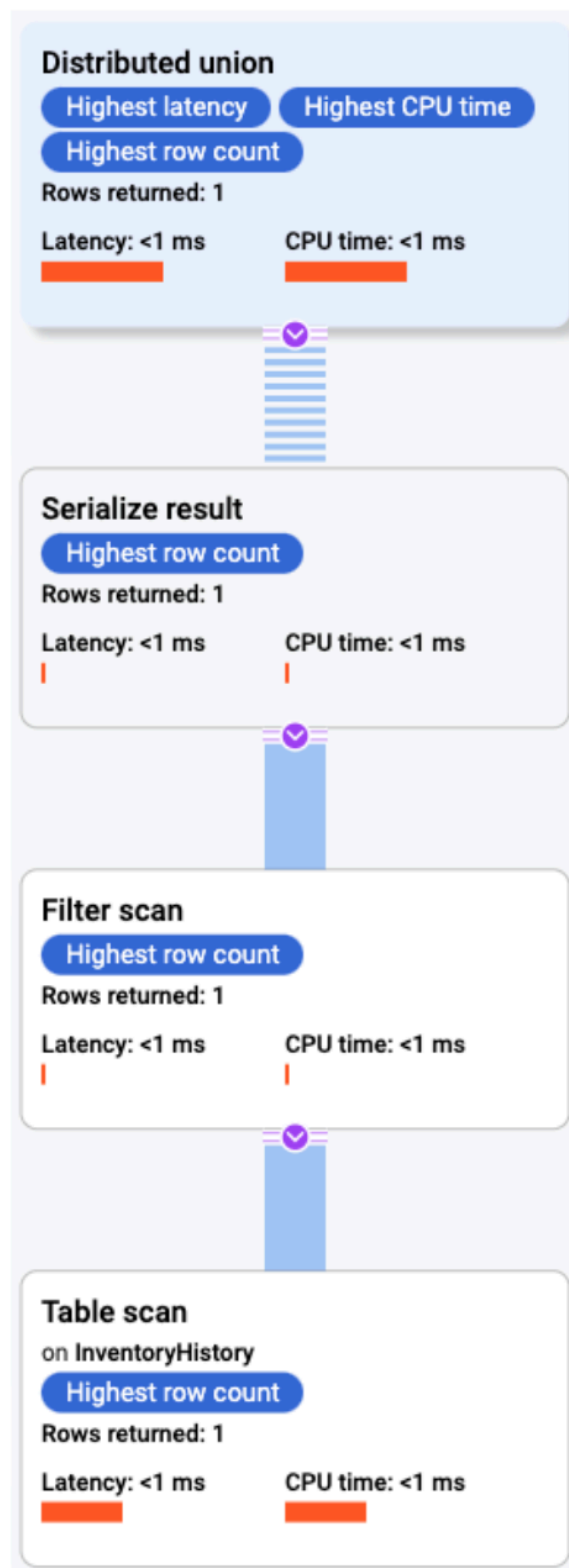
```
SELECT * FROM InventoryHistory WHERE ItemID=1
```

6. 按一下「執行」
7. 按一下「說明」

Cloud 控制台會顯示查詢的視覺化執行計畫。



Expanded



就概念上而言，執行計畫就是相關運算子的樹狀結構。每個運算子都會從輸入讀取資料列，並產生輸出的資料列。系統會傳回執行作業根部的運算子結果做為 SQL 查詢的結果。

查詢最佳化工具

Cloud Spanner 查詢最佳化工具會比較替代執行計畫，然後選取最有效率的計畫。隨著時間推移，查詢最佳化工具會不斷演進，擴大查詢執行計畫的選擇範圍，並提高估計值的準確率，進而產生更有效率的查詢執行計畫。

Cloud Spanner 會以新版查詢最佳化工具的形式，推出最佳化工具更新。根據預設，每個資料庫都會在最新版最佳化工具發布後 30 天內，開始使用該版本。

如要在 gcloud spanner 中執行查詢時查看所用版本，請將 `--query-mode` 旗標設為 PROFILE

1. 輸入下列指令即可查看最佳化工具版本

```
gcloud spanner databases execute-sql $SPANNER_DB --instance=$SPANNER_INSTANCE \  
--query-mode=PROFILE --sql='SELECT * FROM InventoryHistory'
```

輸出範例

```

TOTAL_ELAPSED_TIME: 6.18 msecs
CPU_TIME: 5.17 msecs
ROWS_RETURNED: 1
ROWS_SCANNED: 1
OPTIMIZER_VERSION: 3
RELATIONAL Distributed Union
(1 execution, 0.11 msecs total latency)
subquery_cluster_node: 1
|
+- RELATIONAL Distributed Union
| (1 execution, 0.09 msecs total latency)
| call_type: Local, subquery_cluster_node: 2
| |
| \- RELATIONAL Serialize Result
| (1 execution, 0.08 msecs total latency)
| |
| +- RELATIONAL Scan
| | (1 execution, 0.08 msecs total latency)
| | Full scan: true, scan_target: InventoryHistory, scan_type: TableScan
| | |
| | +- SCALAR Reference
| | | ItemRowID
| | |
| | +- SCALAR Reference
| | | ItemID
| | |
| | +- SCALAR Reference
| | | InventoryChange
| | |
| | \- SCALAR Reference
| | | Timestamp
| | |
| +- SCALAR Reference
| | $ItemRowID
| |
| +- SCALAR Reference
| | $ItemID
| |
| +- SCALAR Reference
| | $InventoryChange
| |
| \- SCALAR Reference
| | $Timestamp
| |
| \- SCALAR Constant
| true

```

ItemRowID: 1


```
ItemID: 1
InventoryChange: 3
Timestamp:
```

目前版本為第 3 版。如要查看最新版本，請參閱[版本記錄](#)

更新最佳化工具版本

本實驗室使用的最新版本為 4 版。接著，您將更新 Spanner 資料表，以便查詢最佳化工具使用第 4 版。

2. 更新最佳化工具

```
gcloud spanner databases ddl update $SPANNER_DB \
--instance=$SPANNER_INSTANCE \
--ddl='ALTER DATABASE InventoryHistory
SET OPTIONS (optimizer_version = 4)'
```

輸出範例

```
Schema updating...done.
```

3. 輸入下列指令，查看最佳化工具版本更新

```
gcloud spanner databases execute-sql $SPANNER_DB --instance=$SPANNER_INSTANCE \
--query-mode=PROFILE --sql='SELECT * FROM InventoryHistory'
```

輸出範例

```
TOTAL_ELAPSED_TIME: 8.57 msecs
CPU_TIME: 8.54 msecs
ROWS_RETURNED: 1
ROWS_SCANNED: 1
OPTIMIZER_VERSION: 4
[...]
```

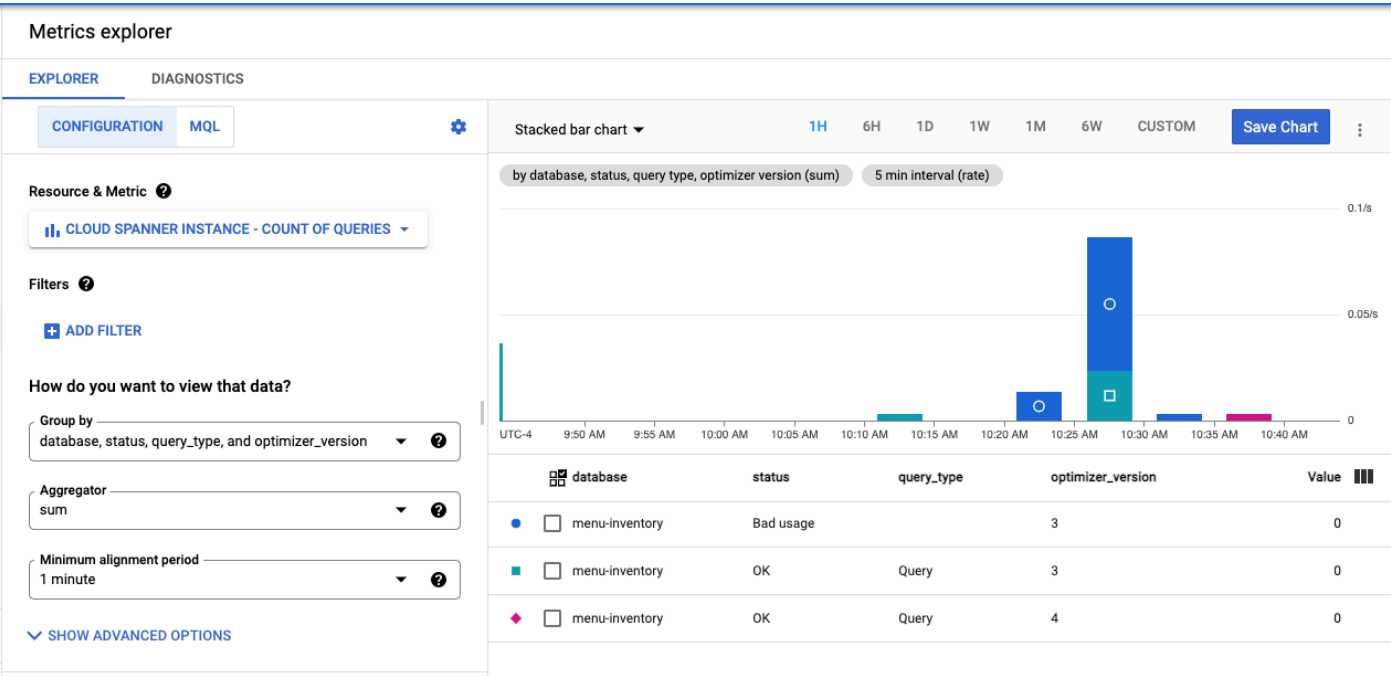
「`OPTIMIZER_VERSION`」已更新至第 4 版

在 Metrics Explorer 中以圖表呈現查詢最佳化工具版本

您可以使用 Cloud Console 中的 Metrics Explorer，以視覺化方式呈現資料庫執行個體的**查詢次數**。您可以查看每個資料庫使用的最佳化工具版本。

1. 前往 Cloud 控制台的 Monitoring，然後在左選單中選取「Metrics Explorer」。
2. 在「資源類型」欄位中，選取 Cloud Spanner 執行個體。
3. 在「指標」欄位中，選取「查詢次數」並套用。

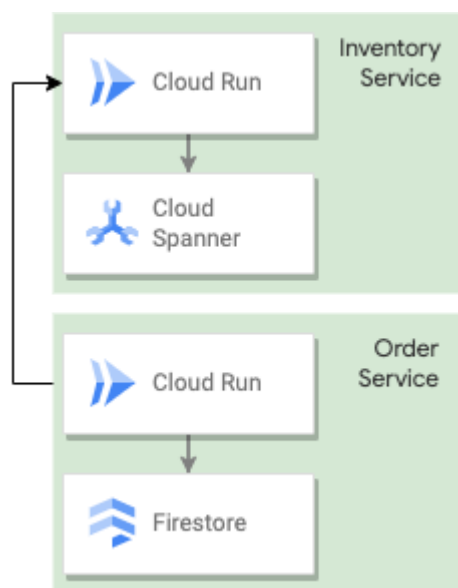
4. 在「Group By」(依據分組) 欄位中，選取資料庫、optimizer_version 和狀態。



7. 建立及設定 Firestore 資料庫

Firestore 是 NoSQL 文件資料庫，專為自動調整資源配置、提供高效能，以及協助輕鬆開發應用程式而打造。雖然 Firestore 介面有許多與傳統資料庫相同的功能，但 NoSQL 資料庫說明資料物件關係的方式與傳統資料庫不同。

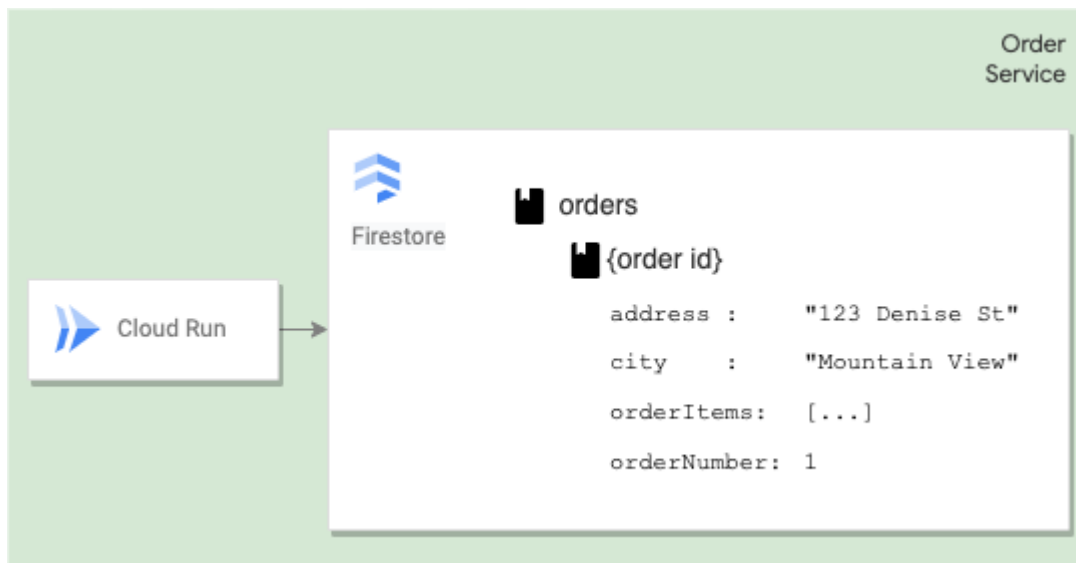
在下列工作中，您將建立由 Firestore 支援的訂購服務 Cloud Run 應用程式。訂購服務會呼叫前一節建立的庫存服務，在開始訂購前查詢 Spanner 資料庫。這項服務可確保有足夠的廣告空間，並完成訂單。



8. Firestore 概念

資料模型

Firestore 資料庫是由集合和文件組成。



文件

每份文件都含有鍵/值組合。Firestore 經過最佳化調整，可儲存大量小型文件。

```
{order id}
  address : "123 Denise St"
  city    : "Mountain View"
  orderItems: [...]
  orderNumber: 1
```

在上述範例中，訂單 ID 文件包含四個鍵值組合。orderItems 鍵包含鍵/值組合陣列。

集合

您必須將所有文件儲存在集合中。文件可以包含子集合和巢狀物件，包括字串等原始欄位，或清單等複雜物件。

```
orders
  {order id}
    address : "123 Denise St"
    city    : "Mountain View"
    orderItems: [...]
    orderNumber: 1
```

在上述範例中，訂單 ID 文件會儲存在訂單集合中。

建立 Firestore 資料庫

1. 建立 Firestore 資料庫

```
gcloud firestore databases create --location=$REGION
```

輸出範例

```
Success! Selected Google Cloud Firestore Native database for cymbal-eats-6422-3462
```

您建立的新 Firestore 資料庫目前是空的。新資料庫也有一組預設安全性規則，允許任何人執行讀取作業，但禁止任何人寫入資料庫。

步驟 9 · 約 0 分鐘

9. 將 Firestore 整合至應用程式

在本節中，您將更新服務帳戶、新增 Firestore 存取服務帳戶、檢查及部署 Firestore 安全性規則，並查看 Firestore 中的資料修改方式。

設定驗證方法

1. 將 Datastore 使用者角色授予服務帳戶

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$PROJECT_NUMBER-compute@developer.gserviceaccount.com" \
  --role="roles/datastore.user"
```

輸出範例

```
Updated IAM policy for project [cymbal-eats-6422-3462].
```

Datastore 使用者角色可授予 Firestore 資料庫的資料讀寫權限。

Firestore 安全性規則

安全性規則提供存取控管和資料驗證功能，格式簡單明瞭。

1. 前往 `order-service/starter-code` 目錄

```
cd ~/cymbal-eats/order-service
```

2. 在 Cloud 編輯器中開啟 `firestore.rules` 檔案

```
cat firestore.rules
```

firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read: if true;
    }
    match /{document=**} {
      allow write: if false;
    }
  }
}
```

firestore.rules 檔案包含的規則允許所有讀取作業，並拒絕 Firestore 資料庫中所有資料表的寫入作業。如要進一步瞭解 Firestore 規則，請參閱「[開始使用安全性規則](#)」。

警告：建議您限制 Firestore 儲存空間的存取權。在本實驗室中，我們允許所有讀取作業。不建議在正式環境中使用此設定。

啟用 Firestore Managed Services

1. 按一下「開啟終端機」
2. 使用目前的專案 ID 建立 .firebaserc 檔案。部署目標的設定會儲存在專案目錄的 .firebaserc 檔案中。

firebaserc.tpl

```
sed "s/PROJECT_ID/$PROJECT_ID/g" firebaserc.tpl > .firebaserc
```

2. 下載 Firebase 二進位檔

```
curl -sL https://firebase.tools | upgrade=true bash
```

輸出範例

```
-- Checking for existing firebase-tools on PATH...
Your machine already has firebase-tools@10.7.0 installed. Nothing to do.
-- All done!
```

3. 部署 Firestore 規則。

```
firebase deploy
```

輸出範例

```
=== Deploying to 'cymbal-eats-6422-3462'...

i  deploying firestore
i  cloud.firestore: checking firestore.rules for compilation errors...
✓  cloud.firestore: rules file firestore.rules compiled successfully
i  firestore: uploading rules firestore.rules...
✓  firestore: released rules firestore.rules to cloud.firestore

✓  Deploy complete!

Project Console: https://console.firebase.google.com/project/cymbal-eats-6422-3462/overview
```

更新 Cloud Firestore 安全性規則後，最多需要一分鐘才會影響新的查詢和監聽器。不過，變更最多可能需要 10 分鐘才會全面生效，並影響所有有效監聽器。

修改資料

Firestore 會隱含建立集合和文件。只要將資料指派給集合中的文件即可。如果集合或文件不存在，Firestore 會建立該項目。

將資料新增至 Firestore

您可以透過多種方式將資料寫入 Cloud Firestore：

- 在集合中設定文件的資料，並明確指定文件 ID。
- 將新文件新增至集合。在這種情況下，Cloud Firestore 會自動產生文件 ID。
- 建立空白文件 (內含自動產生的 ID)，稍後再指派資料。

下一節將說明如何使用 `set` 方法建立文件。

設定文件

使用 `set()` 方法建立文件。使用 `set()` 方法時，您必須指定要建立的文件 ID。

請參閱下方的程式碼片段。

index.js

```
const orderDoc = db.doc(`orders/123`);
await orderDoc.set({
  orderNumber: 123,
  name: Anne,
  address: 555 Bright Street,
  city: Mountain View,
  state: CA,
  zip: 94043,
  orderItems: [id: 1],
  status: 'New'
});
```

這段程式碼會建立文件，並指定使用者產生的文件 ID 123。如要讓 Firestore 代表您產生 ID，請使用 `add()` 或 `create()` 方法。

使用 `set()` 時，如果文件不存在，系統會建立文件。如果文件存在，系統會以新提供的資料覆寫文件內容。

更新文件

您可以使用 `update()` 更新部分文件欄位，不必覆寫整份文件。

在下列程式碼片段中，程式碼會更新訂單 123

index.js

```
const orderDoc = db.doc(`orders/123`);  
await orderDoc.update(name: "Anna");
```

刪除文件

在 Firestore 中，您可以刪除集合、文件或文件中的特定欄位。如要刪除文件，請使用 `delete()` 方法。

以下程式碼片段會刪除訂單 123。

index.js

```
const orderDoc = db.doc(`orders/123`);  
await orderDoc.delete();
```

注意：刪除文件不會刪除其子集合！

10. 部署及測試

在本節中，您將應用程式部署至 Cloud Run，並測試建立、更新和刪除方法。

將應用程式部署至 Cloud Run

1. 將網址儲存在 INVENTORY_SERVICE_URL 變數中，以便與 Inventory Service 整合

```
INVENTORY_SERVICE_URL=$(gcloud run services describe inventory-service \
  --region=$REGION \
  --format=json | jq \
  --raw-output ".status.url")
```

訂單服務必須與庫存服務通訊，確認庫存存在，且訂單可以履行。在這個步驟中，您會將庫存服務的網址儲存至變數，並以環境變數的形式傳遞至訂單服務。

2. 部署訂單服務

```
gcloud run deploy order-service \
  --source . \
  --platform managed \
  --region $REGION \
  --allow-unauthenticated \
  --project=$PROJECT_ID \
  --set-env-vars=INVENTORY_SERVICE_URL=$INVENTORY_SERVICE_URL \
  --quiet
```

輸出範例

```
[...]
Done.
Service [order-service] revision [order-service-00001-qot] has been deployed and is
serving 100 percent of traffic.
Service URL: https://order-service-3jbm3exegq-uk.a.run.app
```

測試 Cloud Run 應用程式

建立文件

1. 將訂單服務應用程式的網址儲存至變數，以供測試

```
ORDER_SERVICE_URL=$(gcloud run services describe order-service \
  --platform managed \
  --region $REGION \
  --format=json | jq \
  --raw-output ".status.url")
```

2. 建立訂單要求，並將新訂單發布至 Firestore 資料庫

```
curl --request POST $ORDER_SERVICE_URL/order \
--header 'Content-Type: application/json' \
--data-raw '{
  "name": "Jane Doe",
  "email": "Jane.Doe-cymbaleats@gmail.com",
  "address": "123 Maple",
  "city": "Buffalo",
  "state": "NY",
  "zip": "12346",
  "orderItems": [
    {
      "id": 1
    }
  ]
}'
```

輸出範例

```
{"orderNumber":46429}
```

儲存訂單編號，稍後會用到

```
export ORDER_NUMBER=<value_from_output>
```

查看結果

在 Firestore 中查看結果

1. 前往 [Firestore 控制台](#)
2. 按一下「資料」

Data			Cloud Firestore in Native mode ⓘ Database location: us-east4
/ > orders > 46429 ✎			
Root	orders		46429
+ START COLLECTION	+ ADD DOCUMENT		+ START COLLECTION
⋮ orders >	⋮ 46429 >		+ ADD FIELD
			address: "123 Maple"
			city: "Buffalo"
			name: "Jane Doe"
			▼ orderItems
			0
			id: 1
			orderNumber: 46429
			placedAt: April 26, 2022 at 11:42:10 PM UTC-4
			state: "NY"
			status: "New"
			statusUpdatedAt: April 26, 2022 at 11:42:10 PM UTC-4
			zip: "12346"

更新文件

提交的訂單未包含數量。

1. 更新記錄並新增數量鍵/值組合

```
curl --location -g --request PATCH $ORDER_SERVICE_URL/order/${ORDER_NUMBER} \
--header 'Content-Type: application/json' \
--data-raw '{
  "orderItems": [
    {
      "id": 1,
      "quantity": 1
    }
  ]
}'
```

輸出範例

```
{"status": "success"}
```

查看結果

在 Firestore 中查看結果

3. 前往 [Firestore 控制台](#)

4. 按一下「資料」

Data			Cloud Firestore in Native mode ⓘ Database location: us-east4
/ > orders > 46429 ✎			
Root	orders		46429
+ START COLLECTION	+ ADD DOCUMENT		+ START COLLECTION
⋮ orders >	⋮ 46429 >		+ ADD FIELD
			address: "123 Maple"
			city: "Buffalo"
			name: "Jane Doe"
			▼ orderItems
			▼ 0
			id: 1
			quantity: 1
			orderNumber: 46429
			placedAt: April 26, 2022 at 11:42:10 PM UTC-4
			state: "NY"
			status: "New"
			statusUpdatedAt: April 26, 2022 at 11:42:10 PM UTC-4
			zip: "12346"

使用 patch() 更新 Firestore 中的 NoSQL 結構時，系統只會更新呼叫中傳遞的項目。

刪除文件

1. 從 Firestore 訂單集中刪除項目 46429

```
curl --location -g --request DELETE $ORDER_SERVICE_URL/order/${ORDER_NUMBER}
```

查看結果

- 5. 前往 [Firestore 控制台](#)
- 6. 按一下「資料」

Data			Cloud Firestore in Native mode ⓘ Database location: us-east4
/ > orders > 46429 ✎			
Root	orders	46429	
+ START COLLECTION	+ ADD DOCUMENT	+ START COLLECTION	
⋮ orders >		+ ADD FIELD	
		There's no document in this path yet	
		Learn more	

文件 46429 已刪除，但訂單集合仍保留。

步驟 11 · 約 0 分鐘

11. 恭喜！

恭喜，您已完成本實驗室！

後續步驟：

探索其他 Cymbal Eats 程式碼研究室：

- [使用 Eventarc 觸發 Cloud Workflows](#)
- [透過 Cloud Storage 觸發事件處理作業](#)
- [從 Cloud Run 連線至私人 Cloud SQL](#)
- [使用 Identity-Aware Proxy \(IAP\) 保護無伺服器應用程式](#)
- [使用 Cloud Scheduler 觸發 Cloud Run 工作](#)
- [安全部署至 Cloud Run](#)
- [保護 Cloud Run 輸入流量](#)
- [從 GKE Autopilot 連線至私人 AlloyDB](#)

清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除含有相關資源的專案，或者保留專案但刪除個別資源。

刪除專案

如要避免付費，最簡單的方法就是刪除您為了本教學課程所建立的專案。
